

构建你自己的 Code-Editing Agent

中文技术手册·四步 200 行·配可运行 Python 实现

思想来源 Thorsten Ball 《How to Build an Agent》·中文手册与代码为另行编写·2026-09-05

目录

来源与版权说明	2
本册结构	3
第一章·为什么是这 200 行	4
一个反直觉的事实	4
定义：什么才算 agent	4
这一册的四步	4
准备	5
第二章·三个前置概念	6
概念一：服务端是无状态的	6
概念二：工具的本质是一个约定	6
概念三：循环的形状	7
第三章·第 1 步：循环（还不是 agent）	8
这一步在干什么	8
三个值得停一下的细节	8
验收	8
第四章·第 2 步：read_file ——chatbot 与 agent 的分界	10
先手工模拟一遍	10
工具的四要素落到代码	10
循环的改造：一个布尔量	11
一个必须做对的协议细节	11
错误也是要回灌的	11
验收	12
第五章·第 3 步：list_files ——工具是会自己组合的	13
实现本身很无聊	13
真正的看点：组合	13
返回格式：一个没有标准答案的设计题	13
验收	14
第六章·第 4 步：edit_file ——完整的 code-editing agent	15
设计：为什么是字符串替换	15
新建文件的约定	15
本册加的一条检查：唯一性	15
验收：两个动作	16
到这里就完了	16

第七章·完整代码清单 17

 pyproject.toml 17

 step1_chat.py 17

 tools.py 18

 agent.py 22

第八章·实测勘误与往外扩 25

 一、照抄会踩的坑（实测过） 25

 坑 1： from jsonschema import generate_schema 这个函数不存在 25

 坑 2： 模型 ID 过期 25

 坑 3（移植特有）： Ctrl-D 退出 25

 二、Go 原版与 Python 版的取舍差异 25

 三、这 200 行往外扩，扩什么 26

 四、接着读什么 26

附录 A·英文原文存档： How to Build an Agent 27

How to Build an Agent 27

 Pencils out! 27

 A First Tool 30

 The read_file tool 32

 The list_files tool 39

 Let it edit_file 42

 Isn't this amazing? 48

附录 B·Medium Python 移植版页面存档 49

来源与版权说明

本册是个人离线学习资料，供本人阅读与跟学，请勿再分发。

项	内容
思想来源	<i>How to Build an Agent</i> , Thorsten Ball （ Amp / Sourcegraph ） , 2025-04-15
原文地址	https://ampcode.com/how-to-build-an-agent
Python 移植参考	Janitha Rathnayake , 2025-08-19 , https://medium.com/@jbrathnayake98/how-to-build-an-agent-by-thorsten-ball-python-version-ebbabb8665f6
本册正文	归档时另行编写的中文技术手册与 Python 实现，非原文翻译
附录 A	Thorsten Ball 英文原文存档，版权归原作者所有
成稿时间	2026-09-05

说明：本册第一至第八章是重新编写的中文技术手册——按原文的四步教学骨架组织，但论证、代码、勘误与延伸都是另行撰写并实测的，不是逐段翻译。需要原作者的完整英文叙述，见附录 A （ 22 个代码块全量存档）；需要 Medium 那一版页面本身，浏览器打印即可。

本册结构

章	内容	你会得到
一	为什么是这 200 行	判断力：什么是 agent，什么不是
二	三个前置概念	无状态服务端 / 工具的本质 / 循环的形状
三	第 1 步：循环	一个会记上下文的终端聊天框
四	第 2 步：read_file	第一个工具，chatbot → agent 的分界
五	第 3 步：list_files	工具组合，你没写编排逻辑
六	第 4 步：edit_file	完整的 code-editing agent
七	完整代码清单	可直接跑的 uv 工程
八	实测勘误与延伸	照抄会踩的坑 + 往外扩的路线
附录 A	英文原文存档	原作者的叙述与 Go 代码

配套代码在 [agent-python-tutorial-src/code/](#)：

pyproject.toml	uv 工程，唯一依赖 anthropic
step1_chat.py	第 1 步：聊天循环
tools.py	三个工具 + 手写 JSON Schema
agent.py	第 2-4 步：完整 agent 循环，--step 控制工具集

第一章·为什么是这 200 行

一个反直觉的事实

看着 agent 改文件、跑命令、报错了自己绕出来、换个策略重试，你会以为背后有什么秘密。

没有。用原作者的话说，它就是「an LLM, a loop, and enough tokens」——一个模型、一个循环、够用的 token。剩下让 Amp、Claude Code 这类产品好用的东西，是 elbow grease（苦功夫），不是什么天才时刻。

这句话值不值得信，最好的验证方式不是读，是自己敲一遍。**200 行、三个工具，你就能拿到一个会读你的代码、会改你的文件的东西。**敲完之后你对”agent”这个词的祛魅是不可逆的。

定义：什么才算 agent

原文给的定义很克制，也很好用：

一个能访问工具的 LLM——因而具备修改上下文窗口之外的东西的能力。

拆开看两个条件：

- 1. **有工具**。没工具的模型只能吐字符串，它改变不了世界任何状态。
- 2. **能改上下文窗口之外的东西**。这是分界线。聊天框里的模型只能改”对话”，agent 能改磁盘、能改数据库、能发请求。

按这个定义反查：

- 一个会调用 `search()` 但把结果只用来润色答复的 RAG 问答——**不是 agent**，它没改外部状态。
- 一个只有 `read_file` 的循环——**是 agent**（勉强），它读了外部状态，尽管没写。
- 一个能 `edit_file` 的循环——**典型 agent**。

这一册的四步

步	加什么	解决什么问题	代码量
1	循环	让模型记住上下文	~40 行
2	<code>read_file</code>	让模型能看见磁盘	~80 行
3	<code>list_files</code>	让模型能自己找路	~100 行
4	<code>edit_file</code>	让模型能改磁盘	~200 行

每一步都能独立跑起来，都有明确的验收动作。**强烈建议真的敲**——不是为了学语法，是为了亲眼看到”这么点代码就够了”这件事，读是读不出来的。

准备

```
cd code
export ANTHROPIC_API_KEY="..."
uv sync
```

Python 3.12+，唯一依赖 [anthropic](#)。原文用 Go，本册用 Python；语言不影响任何结论，循环的形状是一样的。

第二章·三个前置概念

动手之前先把三件事想清楚，后面每一步都在反复用它们。

概念一：服务端是无状态的

这是最多人栽跟头的地方。

你调 `messages.create()` 的时候，Anthropic 的服务端不记得你上一轮说过什么。它只看到你这一次发过去的 `messages` 列表。所谓“Claude 记得我叫什么”，纯粹是因为你本地那个 `conversation` 列表里还留着第一轮的内容，而且你又整个发了一遍。

三个直接推论：

1. 上下文管理从第一行代码起就是你的责任。不是某个高级特性，是循环的固有属性。
2. 每轮的输入是海量重发的。对话越长，每轮越贵、越慢。这就是为什么后面所有 `harness` 都要做压缩、修剪、缓存——它们管的都是这一个列表。
3. 你可以任意篡改历史。删掉某轮、改写某轮、注入一条不存在的“助手说过的话”，模型完全不知情。这既是上下文工程的全部自由度，也是它全部的危险。

概念二：工具的本质是一个约定

原文那个比喻是我见过最好的：

想象你跟朋友说：「接下来的对话里，你想让我抬手的话，就眨一下眼。」

古怪，但一听就懂。工具调用就是这个：

1. 你事先告诉模型有哪些“眨眼”方式（工具名 + 描述 + 参数格式）。
2. 模型想用的时候就眨眼（回复里出现 `tool_use` 块，而不是普通文本）。
3. 你负责抬手（真正执行那个函数）。
4. 你把抬手的结果告诉它（把 `tool_result` 塞回对话，再问一遍）。

就这四步。API 里的 `tools` 参数只是把这套约定标准化了，底下传给模型的仍然是文字。你完全可以不用 `tools` 参数、纯靠 `system prompt` 约定一种格式自己解析——原文就演示过，效果也不差。

所以工具不是“模型的能力”，是你的能力。模型只会眨眼，抬手的一直是你的代码。这句话在后面设计权限和沙箱时会一再回来。

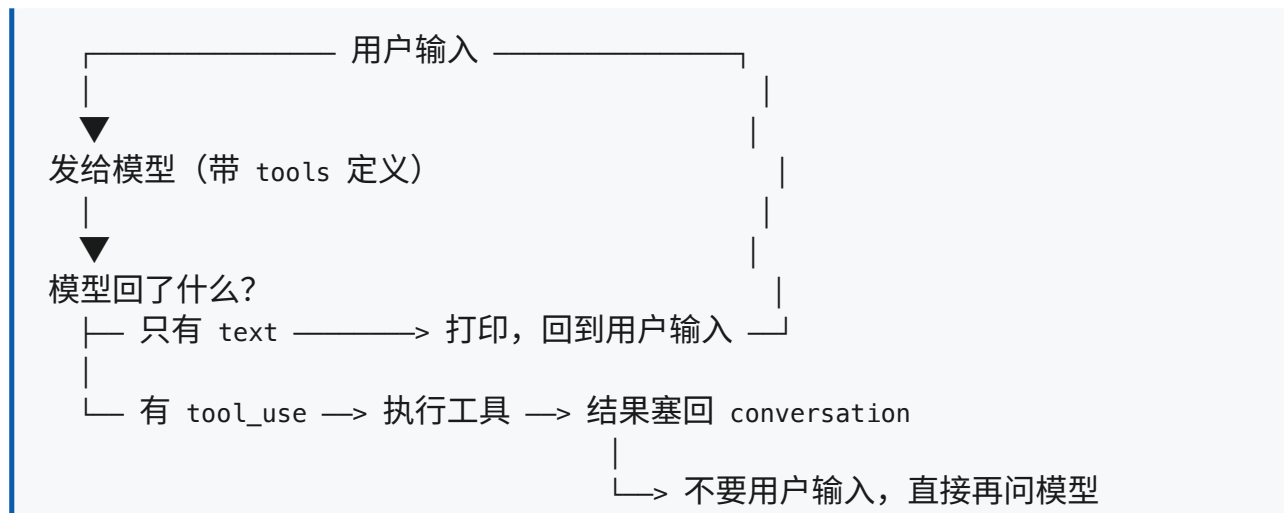
一个工具需要且仅需要四样东西：

要素	给谁看	作用
<code>name</code>	模型	眨眼的暗号
<code>description</code>	模型	什么时候该用、什么时候不该用
<code>input_schema</code>	模型	参数长什么样（JSON Schema）
执行函数	你	真正干活的那一段

其中 `description` 是唯一你能持续调优的旋钮，而且它的效果远超大多数人的预期——第四章会具体说。

概念三：循环的形状

整个 agent 就是下面这一张图，四步之后不会再变，只会往上挂东西：



注意最后那条回边：执行完工具后不回到用户，而是直接再问模型一次。这是 chatbot 和 agent 在代码上的唯一区别——一个 `read_user_input` 布尔量。模型因此可以连续调多个工具、自己串起一条链，中间不需要你插话。

后面所有复杂度——子 agent、审批门、上下文压缩、验证契约——都是挂在这张图的某条边上的中间件。图本身不变。

第三章·第 1 步：循环（还不是 agent）

```
uv run step1_chat.py
```

这一步在干什么

四十行，一个终端聊天框。没有工具，所以按第一章的定义它还不是 **agent**——它改不了上下文窗口之外的任何东西。

但循环的骨架已经齐了：

```
conversation: list[dict] = []
while True:
    user_input = input(...)
    conversation.append({"role": "user", "content": user_input})
    message = client.messages.create(model=MODEL, max_tokens=1024,
    ↪ messages=conversation)
    conversation.append({"role": "assistant", "content": message.content})
    for block in message.content:
        if block.type == "text":
            print(block.text)
```

三个值得停一下的细节

1. **conversation** 里存的是原始 **content** 块，不是字符串。

message.content 是一个块列表（**TextBlock**、后面还有 **ToolUseBlock**）。这里直接原样 **append**。很多实现会图省事写成 `"".join(b.text for b in ...)`，第 1 步看不出问题，到第 2 步加了工具就会炸——因为 **tool_use** 块被你压扁成文本之后，就再也无法和后面的 **tool_result** 用 **tool_use_id** 配对了。从第一行就存对，省得返工。

2. **max_tokens** 是输出上限，不是上下文上限。

第 1 步给 1024 够用。到第 4 步模型要吐整个文件内容，这个值必须调大（代码里是 4096）。调小了的症状很隐蔽：模型的回复会被截断在半句话或者半个 JSON 参数上，看起来像“模型犯傻了”。

3. 退出要处理 **EEOFError**。

Python 的 **input()** 遇到 Ctrl-D 抛 **EEOFError**，不接住的话程序会带着一段丑陋的 **traceback** 退出。Go 版本用 **scanner.Scan()** 返回 **false**，不存在这个问题——这是移植时最容易漏的一处。

验收

连问两轮，第二轮引用第一轮的信息：

You：我叫老周。

Claude: 你好, 老周! ...

You: 我刚才说我叫什么?

Claude: 你说你叫老周。

它”记得”，是因为第二次请求把第一轮的四条消息全都又发了一遍。把 `conversation.append` 那两行注释掉再试一次，你会看到它立刻失忆——这个对照实验值得做一次，比读十遍”服务端无状态”都管用。

第四章·第 2 步：read_file —— chatbot 与 agent 的分界

```
uv run agent.py --step 2
```

先手工模拟一遍

在写任何工具代码之前，值得用第 1 步那个纯聊天程序做一次实验。直接跟模型说：

你是天气专家。我问你某地天气时，你就回复 `get_weather(< 地名 >)`。我会告诉你那里的天气。

然后问它慕尼黑天气。它会老老实实回 `get_weather(Munich)`。你手动回一句“闷热，28 度”，它接着就能正常聊下去。

整个工具调用机制，本质上就是这段对话被标准化了。做过这个实验，后面所有 `tools=[...]` 的代码你都不会觉得是黑魔法。

工具的四要素落到代码

见 `code/tools.py`。以 `read_file` 为例：

```
read_file = Tool(
    name="read_file",
    description=(
        "Read the contents of a given relative file path. "
        "Use this when you want to see what's inside a file. "
        "Do not use this with directory names."
    ),
    input_schema={
        "type": "object",
        "properties": {
            "path": {"type": "string",
                    "description": "The relative path of a file in the working
↪ directory."}
        },
        "required": ["path"],
    },
    run=_read_file,
)
```

注意 `description` 里那句 **“Do not use this with directory names”**。这类否定式约束的性价比极高：不写的话模型会拿目录名调 `read_file`，拿到一个 `IsADirectoryError`，浪费一整轮。工具描述里最值钱的往往不是“能干什么”，是“不要用来干什么”。

循环的改造：一个布尔量

第 1 步到第 2 步，循环只多了一件事——判断模型有没有眨眼：

```
need_user_input = True
while True:
    if need_user_input:
        ... 读用户输入 ...
    message = client.messages.create(..., tools=schemas)
    conversation.append({"role": "assistant", "content": message.content})

    results = []
    for block in message.content:
        if block.type == "text":
            print(block.text)
        elif block.type == "tool_use":
            results.append(self._execute(block))

    if not results:
        need_user_input = True      # 没工具调用 → 回到用户
        continue
    conversation.append({"role": "user", "content": results})
    need_user_input = False        # 有工具调用 → 直接再问模型
```

就是这个 `need_user_input`。它是 chatbot 和 agent 在代码层面的全部差别。

一个必须做对的协议细节

工具结果要以 `tool_result` 块的形式、放在 **user** 角色的消息里回灌：

```
{"type": "tool_result", "tool_use_id": block.id, "content": content, "is_error":
↪ False}
```

三条硬要求：

- `tool_use_id` 必须和模型给的 `tool_use` 块的 `id` 一一对应，缺一个 API 直接报错；
- 整个 `message.content` 要原样回灌，不要 `json.dumps` 成字符串——序列化了配对就断了，还会破坏 prompt caching；
- 一轮里模型可能同时发起多个 `tool_use`，结果要一次性放在同一条 user 消息的列表里，不能拆成多条。

错误也是要回灌的

```
except ToolError as e:
    content, is_error = str(e), True
```

工具失败不要抛给用户，要包成 `is_error: true` 回灌给模型。模型看到“文件不存在”会自己去 `list_files` 找对的路径再重试——这就是最原始的反馈循环，一个 harness 里最有价值的东西，在这里只花了两行。

验收

```
echo '什么动物最不好相处？因为它总说 neigh 。' > secret.txt
uv run agent.py --step 2
```

然后问：「帮我解开 secret.txt 里的谜题」。

你没有说“去读文件”，也没说“如果看到文件名就去打开它”。它自己判断出要读、读了、答了。这一刻就是分界线——从这里开始，你写的不再是一个聊天程序。

第五章·第 3 步：list_files ——工具是会自己组合的

```
uv run agent.py --step 3
```

实现本身很无聊

`os.walk` 走一遍，返回 JSON 数组，目录带尾斜杠。二十行。

```
def _list_files(args):
    root = args.get("path") or "."
    out = []
    for dirpath, dirnames, filenames in os.walk(root):
        dirnames[:] = [d for d in dirnames if d not in {".git", "__pycache__",
↪ ".venv", "venv"}]
        ...
    return json.dumps(sorted(out), ensure_ascii=False)
```

那句 `dirnames[:] = [...]` 是原文没有的。不剪枝的话，在任何真实仓库里第一次调用就会把 `.git/` 下几千个对象文件全倒进上下文——一个工具就能把窗口填满。原地修改 `dirnames` 是 `os.walk` 的剪枝约定，写成 `dirnames = [...]` 是无效的，这是个经典陷阱。

真正的看点：组合

加完这个工具，问一句：

这个目录里的 Python 文件都在干什么？简短点。

观察它的行为：先 `list_files({})` 看有什么，再挑出 `.py` 文件 `read_file` 逐个读，然后总结。

你一行编排逻辑都没写。没有 `if`、没有状态机、没有“先列目录再读文件”的提示词。它自己把两个工具串起来了，因为第二章那条回边让它可以连续调用而不必回到用户。

这是 `agent` 这个范式真正的杠杆所在：你提供能力，它提供编排。你要做的是把每个工具的边界和描述写清楚，而不是替它设计流程。这也解释了为什么后面 `harness` 的工作重心是“设计环境”而不是“写流程”。

返回格式：一个没有标准答案的设计题

`list_files` 该返回什么格式？原文的处理是返回字符串列表、目录加尾斜杠，并且明确说了这是随手定的，没有固定格式。

其他可选项都成立：

- 每个目录前缀 `directory:`
- 返回 Markdown，两个二级标题分别是 `directories` 和 `files`
- 返回带缩进的树状文本

选哪个取决于三件事：**模型读得顺不顺**、**花多少 token**、**生成和解析快不快**。这三件事只能试出来，不能推出来。

这里有个更一般的原则值得记住：**工具的返回格式是提示词工程的一部分，不是数据序列化问题**。你不是在设计 API 给程序用，是在设计一段文字给一个会读文字的东西看。JSON 不一定比自然语言好——很多时候恰恰相反。

验收

一句话触发两个工具的连续调用，中间没有你的输入。看终端里连着打印出两行 `tool:` 就对了。

第六章·第 4 步：edit_file ——完整的 code-editing agent

```
uv run agent.py --step 4
```

到这一步你会期待一个复杂的实现：AST 操作？diff/patch 算法？行号定位？

都不是。是字符串替换。

设计：为什么是字符串替换

```
def _edit_file(args):
    path, old, new = args["path"], args["old_str"], args["new_str"]
    ...
    content.replace(old, new)
```

给一个 `old_str` 和一个 `new_str`，在文件里替换。原文的原话是这个决定的最好注脚：Claude 「loves replacing strings」——而且这类偏好是试出来的，不是推出来的。

为什么它比看起来更合理：

- 模型天然擅长复制粘贴一段它刚读过的文本，不擅长数行号（行号会随编辑漂移，是 agent 编辑类工具的经典错误来源）；
- `old_str` 同时起到了定位和校验两个作用——如果文件在模型读过之后被改动了，`old_str` 匹配不上，编辑会失败而不是改错地方；
- 实现只有二十行，不引入任何依赖。

新建文件的约定

`old_str` 为空字符串时，约定为“新建文件”：

```
if old == "":
    os.makedirs(os.path.dirname(path), exist_ok=True) # 父目录不存在时补上
    open(path, "w").write(new)
    return f"Successfully created file {path}"
```

一个工具承担两种语义，不算漂亮，但省掉一个工具定义就是省掉一份上下文和一次模型的选择困难。在工具数量上做减法，通常比在工具能力上做加法更划算。

本册加的一条检查：唯一性

原文的实现直接 `strings.Replace(..., -1)`，替换所有命中。本册改成：

```
occurrences = content.count(old)
if occurrences == 0:
```

```
raise ToolError("old_str not found in file")
if occurrences > 1:
    raise ToolError(f"old_str matched {occurrences} times; make it unique")
```

理由值得展开，因为它是整册最重要的一条工程判断：

模型给出的 `old_str` 经常是一小段常见代码，比如 `return None` 或者 `}`。文件里命中三处的话，全量替换会同时改坏三个地方，然后返回“OK”。你得到一个静默的错误——工具报告成功，代码坏了，模型以为自己做对了，继续往下走。

而报错的版本会发生什么？模型收到 `matched 3 times; make it unique`，自己会把 `old_str` 扩充几行让它唯一，然后重试。一次往返，问题解决。

给 **agent** 一个会失败的工具，远好过给它一个会悄悄做错的工具。

这句话是后面所有 **guardrail** 设计的种子。第八章会接着往下说。

验收：两个动作

动作一，让它创建并跑起来：

建一个 `fizzbuzz.js`，能用 `node` 跑，里面有 `fizzbuzz` 并且会执行。

看它 `edit_file` 建文件，然后你 `node fizzbuzz.js` 验证输出对。

动作二，让它改：

把 `fizzbuzz.js` 改成只打印到 15。

这一步比第一步有意思得多。观察它：

1. 先 `read_file` 把当前内容读回来（它知道自己需要 `old_str` 精确匹配）；
2. `edit_file` 改掉调用参数 `fizzBuzz(100)` → `fizzBuzz(15)`；
3. 再 `edit_file` 一次，把文件顶部注释里的“1 to 100”也改成“1 to 15”。

第 3 步没有人要求它做。它自己认为注释和代码不一致是个问题。这种“顺手把相关的地方一起改了”的行为，是这 200 行里最像一个真实同事的瞬间。

到这里就完了

约 200 行，三个工具，一个循环。一个能读、能找、能改你代码的 **agent**。

原文那句总结值得原样记住：

These models are incredibly powerful now. 300 lines of code and three tools and now you're able to talk to an alien intelligence that edits your code.

剩下的一切——编辑器集成、系统提示词调优、更好的工具、多 **agent**、UI——用原作者的说法都是 **practical engineering and elbow grease**，没有天才时刻。

第七章·完整代码清单

本章是 `agent-python-tutorial-src/code/` 的全量清单，可直接照着敲或直接跑。

```
cd code && export ANTHROPIC_API_KEY="..." && uv sync
uv run agent.py --step 4
```

pyproject.toml

uv 工程定义（9 行）

```
[project]
name = "code-editing-agent"
version = "0.1.0"
description = " 跟学 Thorsten Ball 《How to Build an Agent》的 Python 实现（四步
↳ 递进） "
requires-python = ">=3.12"
dependencies = ["anthropic>=0.40"]

[tool.uv]
package = false
```

step1_chat.py

第 1 步：聊天循环（还不是 agent）（38 行）

```
""" 第 1 步：一个会记住上下文的终端聊天循环。

对应原文 "Let's dive right in" 一节。这一步还不是 agent ——它只有循环，没有工具
↳ 。
要点： Anthropic 的服务端是无状态的，它只看到你这次发过去的 messages 列表，
所以" 记忆" 完全是本地这个 conversation 列表维持出来的。
"""

import anthropic

MODEL = "claude-sonnet-5"

def main() -> None:
    client = anthropic.Anthropic() # 读环境变量 ANTHROPIC_API_KEY
    conversation: list[dict] = []

    print("Chat with Claude (Ctrl-D 退出)")
    while True:
```

```

try:
    user_input = input("\033[94mYou\033[0m: ")
except EOFError:
    break
if not user_input:
    continue

conversation.append({"role": "user", "content": user_input})
message = client.messages.create(
    model=MODEL, max_tokens=1024, messages=conversation
)
conversation.append({"role": "assistant", "content": message.content})

for block in message.content:
    if block.type == "text":
        print(f"\033[93mClaude\033[0m: {block.text}")

if __name__ == "__main__":
    main()

```

tools.py

三个工具 + 手写 JSON Schema （ 166 行）

""" 工具定义与三个内置工具： *read_file / list_files / edit_file* 。

原文用 Go 的结构体 + 反射生成 JSON Schema ；某些 Python 移植版写成

``from jsonschema import generate_schema``——那个函数在 *jsonschema* 包里并不存在（实
↪ 测

4.26.0 无此符号），照抄会直接 *ImportError* 。这里直接手写 *schema* ，零额外依赖，也
↪ 更透明：

模型看到的本来就只是这段 JSON 。

"""

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
from dataclasses import dataclass
```

```
from typing import Any, Callable
```

```
@dataclass(frozen=True)
```

```
class Tool:
```

```
    name: str
```

```

description: str
input_schema: dict[str, Any]
run: Callable[[dict[str, Any]], str]

def to_api(self) -> dict[str, Any]:
    return {
        "name": self.name,
        "description": self.description,
        "input_schema": self.input_schema,
    }

class ToolError(Exception):
    """ 工具执行失败。会被包成 is_error 的 tool_result 回灌给模型，让它自己纠错
    ↪ 。 """

# --- read_file -----

def _read_file(args: dict[str, Any]) -> str:
    path = args["path"]
    try:
        with open(path, encoding="utf-8") as f:
            return f.read()
    except OSError as e:
        raise ToolError(str(e)) from e

read_file = Tool(
    name="read_file",
    description=(
        "Read the contents of a given relative file path. "
        "Use this when you want to see what's inside a file. "
        "Do not use this with directory names."
    ),
    input_schema={
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "The relative path of a file in the working
    ↪ directory.",
            }
        },
        "required": ["path"],
    },
    run=_read_file,

```

```

)

# --- list_files -----

def _list_files(args: dict[str, Any]) -> str:
    root = args.get("path") or "."
    try:
        out: list[str] = []
        for dirpath, dirnames, filenames in os.walk(root):
            dirnames[:] = [d for d in dirnames if d not in {".git", "__pycache__",
↪ ".venv", "venv"}]
            rel_dir = os.path.relpath(dirpath, root)
            if rel_dir != ".":
                out.append(rel_dir + "/")
            for name in filenames:
                out.append(name if rel_dir == "." else os.path.join(rel_dir, name))
        return json.dumps(sorted(out), ensure_ascii=False)
    except OSError as e:
        raise ToolError(str(e)) from e

list_files = Tool(
    name="list_files",
    description=(
        "List files and directories at a given path. "
        "If no path is provided, lists files in the current directory. "
        "Directories are returned with a trailing slash."
    ),
    input_schema={
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "Optional relative path. Defaults to the current
↪ directory.",
            }
        },
        "required": [],
    },
    run=_list_files,
)

# --- edit_file -----

def _edit_file(args: dict[str, Any]) -> str:

```

```

path, old, new = args["path"], args["old_str"], args["new_str"]
if old == new:
    raise ToolError("old_str and new_str must differ")

if old == "": # 空 old_str 约定为「新建文件」
    parent = os.path.dirname(path)
    if parent:
        os.makedirs(parent, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        f.write(new)
    return f"Successfully created file {path}"

try:
    with open(path, encoding="utf-8") as f:
        content = f.read()
except OSError as e:
    raise ToolError(str(e)) from e

occurrences = content.count(old)
if occurrences == 0:
    raise ToolError("old_str not found in file")
if occurrences > 1:
    # 原文没做这个检查。不做的话模型一次替换会命中多处，改坏文件还沉默通过
    ↪ ○
    raise ToolError(f"old_str matched {occurrences} times; make it unique")

with open(path, "w", encoding="utf-8") as f:
    f.write(content.replace(old, new))
return "OK"

edit_file = Tool(
    name="edit_file",
    description=(
        "Make edits to a text file.\n\n"
        "Replaces 'old_str' with 'new_str' in the given file. "
        "'old_str' and 'new_str' MUST be different from each other, and 'old_str' "
        "must match exactly one place in the file.\n\n"
        "If the file specified with path doesn't exist and 'old_str' is empty, "
        "it will be created."
    ),
    input_schema={
        "type": "object",
        "properties": {
            "path": {"type": "string", "description": "The path to the file"},
            "old_str": {
                "type": "string",

```

```

        "description": "Text to search for – must match exactly and appear
↪ exactly once",
        },
        "new_str": {"type": "string", "description": "Text to replace old_str
↪ with"},
        },
        "required": ["path", "old_str", "new_str"],
    },
    run=_edit_file,
)

ALL_TOOLS = [read_file, list_files, edit_file]

```

agent.py

第 2–4 步：完整 agent 循环（99 行）

""" 第 2–4 步：把聊天循环变成 agent 。

```

uv run agent.py --step 2    # + read_file
uv run agent.py --step 3    # + list_files
uv run agent.py --step 4    # + edit_file （完整版）

```

整个 agent 就是这一个循环：

用户输入 → 模型 → 模型说要用工具？ → 执行 → 结果回灌 → 再问模型 → …
 、没有工具调用 → 打印答复，回到用户输入

"""

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import anthropic
```

```
from tools import ALL_TOOLS, ToolError
```

```
MODEL = "claude-sonnet-5"
```

```
MAX_TOKENS = 4096
```

```
class Agent:
```

```

    def __init__(self, client: anthropic.Anthropic, tools: list) -> None:
        self.client = client
        self.tools = {t.name: t for t in tools}

```

```

def _execute(self, block) -> dict:
    """ 执行一次工具调用, 返回 tool_result 块。失败也返回, 交给模型自己纠错
    """
    print(f"\033[92mtool\033[0m: {block.name}({json.dumps(block.input,
    ensure_ascii=False)})")
    tool = self.tools.get(block.name)
    try:
        if tool is None:
            raise ToolError(f"unknown tool: {block.name}")
        content = tool.run(block.input)
        is_error = False
    except ToolError as e:
        content, is_error = str(e), True
        print(f"\033[91m → error\033[0m: {content}")
    return {
        "type": "tool_result",
        "tool_use_id": block.id,
        "content": content,
        "is_error": is_error,
    }

def run(self) -> None:
    conversation: list[dict] = []
    schemas = [t.to_api() for t in self.tools.values()]
    print(f"Chat with Claude — {len(schemas)} 个工具可用 (Ctrl-D 退出)")

    need_user_input = True
    while True:
        if need_user_input:
            try:
                user_input = input("\033[94mYou\033[0m: ")
            except EOFError:
                break
            if not user_input:
                continue
            conversation.append({"role": "user", "content": user_input})

        message = self.client.messages.create(
            model=MODEL, max_tokens=MAX_TOKENS,
            messages=conversation, tools=schemas,
        )
        # 注意: 整个 content 列表原样回灌, 不要自己序列化成字符串——
        # tool_use / tool_result 的配对是靠这些结构化块成立的。
        conversation.append({"role": "assistant", "content": message.content})

    results = []

```

```
        for block in message.content:
            if block.type == "text":
                print(f"\033[93mClaude\033[0m: {block.text}")
            elif block.type == "tool_use":
                results.append(self._execute(block))

        if not results:
            need_user_input = True
            continue

        conversation.append({"role": "user", "content": results})
        need_user_input = False

def main() -> None:
    p = argparse.ArgumentParser(description=" 跟学版 code-editing agent")
    p.add_argument("--step", type=int, choices=[2, 3, 4], default=4,
                    help="2=read_file, 3=list_files, 4=edit_file")
    args = p.parse_args()

    tools = ALL_TOOLS[: args.step - 1]
    Agent(anthropic.Anthropic(), tools).run()

if __name__ == "__main__":
    main()
```

第八章·实测勘误与往外扩

一、照抄会踩的坑（实测过）

跟学时如果对网上流传的 Python 移植版敲，有两处直接跑不起来。归档时都在本机验证过。

坑 1：`from jsonschema import generate_schema` 这个函数不存在

流传的 Python 版用它来生成工具的 input schema。实测：

```
$ uv run --with jsonschema python -c "import jsonschema;
↪ print(hasattr(jsonschema, 'generate_schema'))"
version: 4.26.0
has generate_schema: False
```

`jsonschema` 这个包是校验用的（`validate`、各版本 `Validator`），它从来没有从 Python 类型生成 schema 的功能。照抄这一行直接 `ImportError`，而且报错信息不会告诉你“这个函数根本不存在”，很容易被误判成版本问题去乱升级。

这个坑的来源不难猜：Go 原版用 `jsonschema` 库配合反射自动生成 schema，移植时按名字直译成了 Python 的同名包，但两者不是一回事。

本册的处理：`tools.py` 里手写 JSON Schema。零额外依赖，而且更透明——模型看到的本来就是这段 JSON，中间加一层生成器只会让你看不清自己到底发了什么给模型。真要自动生成，用 `pydantic` 的 `model_json_schema()`。

坑 2：模型 ID 过期

原文写于 2025-04，用的是 `claude-3-7-sonnet-latest`。现在应换成当前模型（本册代码用 `claude-sonnet-5`）。只是 `agent.py` 顶部一个 `MODEL` 常量。

坑 3（移植特有）：`Ctrl-D` 退出

Go 版靠 `scanner.Scan()` 返回 `false` 优雅退出。Python 的 `input()` 遇到 EOF 抛 `EEOFError`，不接住就是一段 `traceback`。本册代码已处理。

二、Go 原版与 Python 版的取舍差异

点	Go 原版	本册 Python 实现	为什么
schema 生成	反射自动生成	手写 JSON	少一层依赖，看得见发出去的东西
<code>edit_file</code> 多处命中	全量替换	报错要求唯一	避免静默改坏文件
<code>list_files</code> 剪枝	无	跳过 <code>.git/</code> <code>__pycache__</code> <code>/venv</code>	否则一次调用填满上下文

点	Go 原版	本册 Python 实现	为什么
工具错误	回灌给模型	同	这是反馈循环的最小形态
工具集	read / list / edit	同	三个就够

三、这 200 行往外扩，扩什么

写完之后你会发现这个 agent 有一堆明显的毛病。每一个毛病，都恰好对应现代 **harness** 的一个组件——这也是这份教程真正的价值：它让你亲身体会到那些组件为什么存在。

你会遇到的毛病	对应组件	现实中的做法
它乱改不该改的文件	约束	沙箱（限制文件系统边界）、危险操作审批门
对话长了就变慢变贵变傻	上下文管理	修剪旧工具结果、压缩、prompt 缓存
它说”改好了”其实没跑测试	反馈 / 验证	验证门：让它必须跑测试并报告真实结果
每次都要重新解释项目约定	指令 / 记忆	仓库里放 AGENTS.md / CLAUDE.md
大任务它一个人做不完	编排	子 agent 委派，各自独立上下文
同一个坏方法反复试	约束	循环检测：同一文件改 N 次就提醒它换思路

按顺序踩这几个坑，再去读任何一本 **harness** 的书或代码，你会发现它们讲的每一件事你都有过对应的痛感。这就是先手写 **200 行** 的意义——不是为了这 200 行，是为了让后面那几千行变得可理解。

四、接着读什么

- 本仓库另一册《**AI Agent Harness 中文教程**》（256 页 / 39 课）——正好从这个循环开始，一层层加到沙箱、子 agent、验证契约、可扩展性。这是最自然的下一站。
- [docs/Anthropic-长时程 Agent-Harness-合订本.pdf](#)——Anthropic 官方两篇工程博客中译，讲长时程 agent 的 harness 怎么迭代和做减法。
- 读源码：Pi（[earendil-works/pi](#)，4 个工具的极简 harness）、Cline SDK（四层分包）、mini-swe-agent（SWE-bench 作者的 model-agnostic 对照组）。
- 附录 A（下一章）：Thorsten Ball 英文原文全文，22 个代码块，Go 实现。中文手册里说”原文这一节”时翻到那里。

附录 A ·英文原文存档: How to Build an Agent

作者 Thorsten Ball (Amp / Sourcegraph) ·2025-04-15 原文: <https://ampcode.com/how-to-build-an-agent> 抓取时间: 2026-09-05 ·个人离线学习归档, 版权归原作者所有, 请勿再分发。

Thorsten Ball//April 15, 2025

How to Build an Agent

or: The Emperor Has No Clothes

It's not that hard to build a fully functioning, code-editing agent.

It seems like it would be. When you look at an agent editing files, running commands, wriggling itself out of errors, retrying different strategies - it seems like there has to be a secret behind it.

There isn't. It's an LLM, a loop, and enough tokens. It's what we've been saying on the **podcast** from the start. The rest, the stuff that makes **Amp** so addictive and impressive? Elbow grease.

But building a small and yet highly impressive agent doesn't even require that. You can do it in less than 400 lines of code, most of which is boilerplate.

I'm going to show you how, right now. We're going to write some code together and go from zero lines of code to "oh wow, this is...a game changer."

I *urge* you to follow along. No, really. You might think you can just read this and that you don't have to type out the code, but it's less than 400 lines of code. I need you to *feel* how little code it is and I want you to see this with your own eyes in your own terminal in your own folders.

Here's what we need:

- **Go**
- **Anthropic API key** that you set as an environment variable, **ANTHROPIC_API_KEY**

Pencils out!

Let's dive right in and get ourselves a new Go project set up in four easy commands:

```
mkdir code-editing-agent
cd code-editing-agent
go mod init agent
touch main.go
```

Now, let's open **main.go** and, as a first step, put a skeleton of things we need in it:

```
package main

import (
    "bufio"
```

```

    "context"
    "fmt"
    "os"

    "github.com/anthropics/anthropic-sdk-go"
)

func main() {
    client := anthropic.NewClient()

    scanner := bufio.NewScanner(os.Stdin)
    getUserMessage := func() (string, bool) {
        if !scanner.Scan() {
            return "", false
        }
        return scanner.Text(), true
    }

    agent := NewAgent(&client, getUserMessage)
    err := agent.Run(context.TODO())
    if err != nil {
        fmt.Printf("Error: %s\n", err.Error())
    }
}

func NewAgent(client *anthropic.Client, getUserMessage func() (string, bool))
↪ *Agent {
    return &Agent{
        client:      client,
        getUserMessage: getUserMessage,
    }
}

type Agent struct {
    client      *anthropic.Client
    getUserMessage func() (string, bool)
}

```

Yes, this doesn't compile yet. But what we have here is an `Agent` that has access to an `anthropic.Client` (which, by default, looks for `ANTHROPIC_API_KEY`) and that can get a user message by reading from stdin on the terminal.

Now let's add the missing `Run()` method:

```

// main.go

func (a *Agent) Run(ctx context.Context) error {

```

```

conversation := []anthropic.MessageParam{}

fmt.Println("Chat with Claude (use 'ctrl-c' to quit)")

for {
    fmt.Print("\u001b[94mYou\u001b[0m: ")
    userInput, ok := a.getUserMessage()
    if !ok {
        break
    }

    userMessage := anthropic.NewUserMessage(anthropic.NewTextBlock(userInput))
    conversation = append(conversation, userMessage)

    message, err := a.runInference(ctx, conversation)
    if err != nil {
        return err
    }
    conversation = append(conversation, message.ToParam())

    for _, content := range message.Content {
        switch content.Type {
        case "text":
            fmt.Printf("\u001b[93mClaude\u001b[0m: %s\n", content.Text)
        }
    }
}

return nil
}

func (a *Agent) runInference(ctx context.Context, conversation
↪ []anthropic.MessageParam) (*anthropic.Message, error) {
    message, err := a.client.Messages.New(ctx, anthropic.MessageNewParams{
        Model:      anthropic.ModelClaude3_7SonnetLatest,
        MaxTokens:  int64(1024),
        Messages:   conversation,
    })
    return message, err
}

```

It's not a lot, is it? 90 lines and the most important thing in them is this loop in `Run()` that lets us talk to Claude. But that's already the heartbeat of this program.

And for a heartbeat, it's pretty straightforward: we first print a prompt, ask the user to type something, add it to the conversation, send it to Claude, add Claude's response to the conversation, print the response, and around we go.

This is every AI chat application you’ve ever used, except it’s in the terminal.

Let’s run it:

```
export ANTHROPIC_API_KEY="this is the last time i will tell you to set this"

# Download the dependencies
go mod tidy
# Run it
go run main.go
```

Then you can just talk to Claude, like this:

\$ go run main.go

Chat with Claude (use 'ctrl-c' to quit)

You: Hey! I'm Thorsten! How are you?

Claude: Hi Thorsten! I'm doing well, thanks for asking. It's nice to meet you. How are you doing today? Is there something I can help you with?

You: Can you come up with any horse-related nicknames that make fun of my first name?

```
Claude: I can try to come up with some playful horse-related nicknames based on
↳ "Thorsten": * Thorough-bred Thorsten * Trotsten * Thorsten the Stallion *
↳ Neighsten * Horseten * Thorsten the Galloper * Saddle-sten * Thorsten of the
↳ Mane * Canter-sten * Thorsteed These are meant to be lighthearted – I hope they
↳ give you a chuckle!
```

You:

Notice how we kept the same conversation going over multiple turns. It remembers my name from the first message. The [conversation](#) grows longer with every turn and we send the whole conversation every time. The server — Anthropic’s server — is stateless. It only sees what’s in the [conversation](#) slice. It’s up to us to maintain that.

Okay, let’s move on, because the nicknames suck and this is not an agent yet. What’s an agent? Here’s [my definition](#): an LLM with *access to tools*, giving it the ability to modify something outside the context window.

A First Tool

An LLM with *access to tools*? What’s a tool? The basic idea is this: you send a prompt to the model that says it should reply in a certain way if it wants to use “a tool”. Then you, as the receiver of that message, “use the tool” by executing it and replying with the result. That’s it. Everything else we’ll see is just abstraction on top of it.

Imagine you’re talking to a friend and you tell them: “in the following conversation, wink if you want

me to raise my arm”. Weird thing to say, but an easy concept to grasp.

We can already try it without changing any of our code.

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: You are a weather expert. When I ask you about the weather in a given location, I want you to reply with 'get_weather(<location_name>)'. I will then tell you what the weather in that location is. Understood?

Claude: I understand your instructions. When you ask me about the weather in a specific location, I will reply with 'get_weather(<location_name>)'. Then you'll provide me with the weather information for that location, and I can discuss it further with you. I'm ready whenever you'd like to ask about the weather somewhere.

You: Hey, what's the weather in Munich?

Claude: get_weather(Munich)

We told Claude to wink with `get_weather` when it wants to know about the weather. The next step is to raise our arm and reply with “result of the tool”:

You: hot and humid, 28 degrees celcius

Claude: Thank you for providing that information. The current weather in Munich is hot and humid at 28 degrees Celsius (about 82 degrees Fahrenheit). These are fairly warm conditions for Munich, especially with the added humidity which can make it feel even warmer than the temperature indicates. If you're in Munich or planning to visit, it would be good to stay hydrated, wear light clothing, and perhaps plan indoor activities during the hottest parts of the day. The combination of heat and humidity can be quite uncomfortable, particularly if you're not accustomed to such weather. Would you like to know about the weather in any other location?

That worked very well, on first try, didn't it?

These models are trained and fine-tuned to use “tools” and they're very eager to do so. By now, 2025, they kinda “know” that they don't know everything and can use tools to get more information. (Of course that's not precisely what's going on, but it's good enough an explanation for now.)

To summarize, all there is to tools and tool use are two things:

1. You tell the model what tools are available
2. When the model wants to execute the tool, it tells you, you execute the tool and send the response up

To make (1) easier, the big model providers have built-in APIs to send tool definitions along.

Okay, now let's build our first tool: `read_file`

The read_file tool

In order to define the `read_file` tool, we're going to use the types that the Anthropic SDK suggests, but keep in mind: under the hood, this will all end up as strings that are sent to the model. It's all “wink if you want me to use `read_file`”.

Each tool we're going to add will require the following:

- A name
- A description to tell the model what the tool does, when to use it, when to not use it, what it returns and so on
- An input schema that describes, as a JSON schema, what inputs this tool expects and in which form
- A function that actually executes the tool with the input the model sends to us and returns the result

So let's add that to our code:

```
// main.go

type ToolDefinition struct {
    Name          string          `json:"name"`
    Description    string          `json:"description"`
    InputSchema    anthropic.ToolInputSchemaParam `json:"input_schema"`
    Function       func(input json.RawMessage) (string, error)
}
```

Now we give our `Agent` tool definitions:

```
// main.go

// `tools` is added here:
type Agent struct {
    client      *anthropic.Client
    getUserMessage func() (string, bool)
    tools       []ToolDefinition
}

// And here:
func NewAgent(
    client *anthropic.Client,
    getUserMessage func() (string, bool),
    tools []ToolDefinition,
) *Agent {
    return &Agent{
        client:      client,
        getUserMessage: getUserMessage,
    }
}
```

```

        tools:      tools,
    }
}

// And here:
func main() {
    // [... previous code ...]
    tools := []ToolDefinition{}
    agent := NewAgent(&client, getUserMessage, tools)

    // [... previous code ...]
}

```

And send them along to the model in `runInference`:

```

// main.go

func (a *Agent) runInference(ctx context.Context, conversation
↪ []anthropic.MessageParam) (*anthropic.Message, error) {
    anthropicTools := []anthropic.ToolUnionParam{}
    for _, tool := range a.tools {
        anthropicTools = append(anthropicTools, anthropic.ToolUnionParam{
            OfTool: &anthropic.ToolParam{
                Name:      tool.Name,
                Description: anthropic.String(tool.Description),
                InputSchema: tool.InputSchema,
            },
        })
    }

    message, err := a.client.Messages.New(ctx, anthropic.MessageNewParams{
        Model:      anthropic.ModelClaude3_7SonnetLatest,
        MaxTokens:  int64(1024),
        Messages:    conversation,
        Tools:       anthropicTools,
    })
    return message, err
}

```

There's a bunch of type shenanigans going on and I'm not too good in Go-with-generics yet so I'm not going to try to explain `anthropic.String` and `ToolUnionParam` to you. But, really, I swear, it's very simple:

We send along our tool definitions, on the server Anthropic then wraps these definitions in [this system prompt](#) (which isn't much), which it adds to our `conversation`, and the model then replies in a specific way if it wants to use that tool.

Alright, so tool definitions are being sent along, but we haven't defined a tool yet. Let's do that and define `read_file`:

```
// main.go

var ReadFileDefinition = ToolDefinition{
    Name:      "read_file",
    Description: "Read the contents of a given relative file path. Use this when
↪ you want to see what's inside a file. Do not use this with directory names.",
    InputSchema: ReadFileInputSchema,
    Function:    ReadFile,
}

type ReadFileInput struct {
    Path string `json:"path" jsonschema_description:"The relative path of a file in
↪ the working directory."`
}

var ReadFileInputSchema = GenerateSchema[ReadFileInput]()

func ReadFile(input json.RawMessage) (string, error) {
    readFileInput := ReadFileInput{}
    err := json.Unmarshal(input, &readFileInput)
    if err != nil {
        panic(err)
    }

    content, err := os.ReadFile(readFileInput.Path)
    if err != nil {
        return "", err
    }
    return string(content), nil
}

func GenerateSchema[T any]() anthropic.ToolInputSchemaParam {
    reflector := jsonschema.Reflector{
        AllowAdditionalProperties: false,
        DoNotReference:          true,
    }
    var v T

    schema := reflector.Reflect(v)

    return anthropic.ToolInputSchemaParam{
        Properties: schema.Properties,
    }
}
```

That's not much, is it? It's a single function, `ReadFile`, and two descriptions the model will see: our `Description` that describes the tool itself ("Read the contents of a given relative file path. ...") and a description of the single input parameter this tool has ("The relative path of a ...").

The `ReadFileInputSchema` and `GenerateSchema` stuff? We need that so that we can generate a JSON schema for our tool definition which we send to the model. To do that, we use the `jsonschema` package, which we need to import and download:

```
// main.go

package main

import (
    "bufio"
    "context"
    // Add this:
    "encoding/json"
    "fmt"
    "os"

    "github.com/anthropics/anthropic-sdk-go"
    // Add this:
    "github.com/inopop/jsonschema"
)
```

Then run the following:

```
go mod tidy
```

Then, in the `main` function, we need to make sure that we use the definition:

```
func main() {
    // [... previous code ...]
    tools := []ToolDefinition{ReadFileDefinition}
    // [... previous code ...]
}
```

Time to try it!

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: what's in main.go?

Claude: I'll help you check what's in the main.go file. Let me read it for you.

You:

Wait, what? Ho, ho, ho, it wants to use the tool! Obviously the output will be slightly different for you, but it certainly sounds like Claude knows that it can read files, right?

The problem is that we don't listen! When Claude winks, we ignore it. We need to fix that.

Here, let me show you how to do that in a single, quick, surprisingly-agile-for-my-age move by replacing our `Agent`'s `Run` method with this:

```
// main.go

func (a *Agent) Run(ctx context.Context) error {
    conversation := []anthropic.MessageParam{}

    fmt.Println("Chat with Claude (use 'ctrl-c' to quit)")

    readUserInput := true
    for {
        if readUserInput {
            fmt.Print("\u001b[94mYou\u001b[0m: ")
            userInput, ok := a.getUserMessage()
            if !ok {
                break
            }

            userMessage :=
↪ anthropic.NewUserMessage(anthropic.NewTextBlock(userInput))
            conversation = append(conversation, userMessage)
        }

        message, err := a.runInference(ctx, conversation)
        if err != nil {
            return err
        }
        conversation = append(conversation, message.ToParam())

        toolResults := []anthropic.ContentBlockParamUnion{}
        for _, content := range message.Content {
            switch content.Type {
            case "text":
                fmt.Printf("\u001b[93mClaude\u001b[0m: %s\n", content.Text)
            case "tool_use":
                result := a.executeTool(content.ID, content.Name, content.Input)
                toolResults = append(toolResults, result)
            }
        }
        if len(toolResults) == 0 {
            readUserInput = true
            continue
        }
    }
}
```

```

    }
    readUserInput = false
    conversation = append(conversation,
↪ anthropic.NewUserMessage(toolResults...))
    }

    return nil
}

func (a *Agent) executeTool(id, name string, input json.RawMessage)
↪ anthropic.ContentBlockParamUnion {
    var toolDef ToolDefinition
    var found bool
    for _, tool := range a.tools {
        if tool.Name == name {
            toolDef = tool
            found = true
            break
        }
    }
    if !found {
        return anthropic.NewToolResultBlock(id, "tool not found", true)
    }

    fmt.Printf("\u001b[92mtool\u001b[0m: %s(%s)\n", name, input)
    response, err := toolDef.Function(input)
    if err != nil {
        return anthropic.NewToolResultBlock(id, err.Error(), true)
    }
    return anthropic.NewToolResultBlock(id, response, false)
}

```

Squint and you'll see that it's 90% boilerplate and 10% that matter: when we get a `message` back from Claude, we check whether Claude asked us to execute a tool by looking for `content.Type == "tool_use"`, if so we hand over to `executeTool`, lookup the tool by name in our local registry, unmarshal the input, execute it, return the result. If it's an error, we flip a boolean. That's it.

(Yes, there is a loop in a loop, but it doesn't matter.)

We execute the tool, send the result back up to Claude, and ask again for Claude's response. Truly: that's *it*. Let me show you.

Mise-en-place, run this:

```

echo 'what animal is the most disagreeable because it always says neigh?' >>
↪ secret-file.txt

```

That creates a `secret-file.txt` in our directory, containing a mysterious riddle.

In that very same directory, let's run our new tool-using agent, and ask it to look at the file:

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: Claude, buddy, help me solve the riddle in the secret-file.txt file

Claude: I'll help you solve the riddle in the secret-file.txt file. Let me first read the contents of this file to see what the riddle is.

```
tool: read_file({"path":"secret-file.txt"})
```

```
Claude: Great! I've found the riddle. The answer to the riddle is: **A horse** The
↪ riddle is playing on wordplay where "neigh" (the sound a horse makes) sounds
↪ like "nay" (a way to say "no" or disagree with something). So horses are
↪ described as disagreeable because they always say "neigh/nay" - implying
↪ they're always disagreeing.
```

You:

Let's take a deep breath and say it together. Ready? Here we go: *holy shit*. You just give it a tool and it...*uses it when it thinks it'll help solve the task*. Remember: we didn't say *anything* about "if a user asks you about a file, read the file". We also didn't say "if something looks like a filename, figure out how to read it". No, none of that. We say "help me solve the thing in this file" and Claude realizes that it can read the file to answer that and off it goes.

Of course, we *can* be specific and really nudge it towards a tool, but it basically does it all on its own:

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: What's going on in main.go? Be brief!

Claude: I'll check the main.go file to see what's going on.

```
tool: read_file({"path":"main.go"})
```

Claude: Based on my review, main.go implements a Claude AI assistant agent that can use tools. Here's a brief summary: - It creates a command-line chat interface with Claude using the Anthropic Go SDK - Implements a tool system with a single "read_file" tool for reading file contents - Manages the conversation flow with Claude, handling both regular text responses and tool use requests - When Claude requests to use a tool, the agent executes it and returns the results - The program loops continuously to maintain a conversation until the user exits The code essentially builds a simple CLI-based chatbot that can interact with the file system when requested by Claude.

You:

Spot on. Okay, now that we know how to make Claude use tools, let's add a few more.

The `list_files` tool

If you're anything like me, the first thing you do when you log into a new computer is to get your bearings by running `ls` — list files.

Let's give Claude the same ability, a tool to list files. And here's the complete implementation of a `list_files` tool:

```
// main.go

var ListFilesDefinition = ToolDefinition{
    Name:      "list_files",
    Description: "List files and directories at a given path. If no path is
↪ provided, lists files in the current directory.",
    InputSchema: ListFilesInputSchema,
    Function:    ListFiles,
}

type ListFilesInput struct {
    Path string `json:"path,omitempty" jsonschema_description:"Optional relative
↪ path to list files from. Defaults to current directory if not provided."`
}

var ListFilesInputSchema = GenerateSchema[ListFilesInput]()

func ListFiles(input json.RawMessage) (string, error) {
    listFilesInput := ListFilesInput{}
    err := json.Unmarshal(input, &listFilesInput)
    if err != nil {
        panic(err)
    }

    dir := "."
    if listFilesInput.Path != "" {
        dir = listFilesInput.Path
    }

    var files []string
    err = filepath.Walk(dir, func(path string, info os.FileInfo, err error) error {
        if err != nil {
            return err
        }

        relPath, err := filepath.Rel(dir, path)
        if err != nil {
            return err
        }
    })
}
```

```

        if relPath != "." {
            if info.IsDir() {
                files = append(files, relPath+"/")
            } else {
                files = append(files, relPath)
            }
        }
        return nil
    })

    if err != nil {
        return "", err
    }

    result, err := json.Marshal(files)
    if err != nil {
        return "", err
    }

    return string(result), nil
}

```

Nothing fancy here: `list_files` returns the list of files and directories in the current folder. There's a thousand optimizations we could (and probably should) make if this were a serious effort, but since I just want to show you what's in the wizard's hat, this is fine.

One thing to note: we return a list of strings and we denote directories with a trailing slash. That's not required, it's just something I just decided to do. There's no fixed format. Anything goes as long as Claude can make sense of it and whether it can you need to figure out by experimentation. You could also prepend each directory with `"directory: "` or return a Markdown document with two headers: `"directories"` and `"files"`. There's a ton of options and which one you chose depends on what Claude can make the most sense of, how many tokens it requires, how fast it is to generate and read, and so on.

Here, we just want to create a small `list_files` tool and the easiest option wins.

Of course we need to tell Claude about `list_files` too:

```

// main.go

func main() {
    // [... previous code ...]

    tools := []ToolDefinition{ReadFileDefinition, ListFilesDefinition}

    // [... previous code ...]

```

}

And that's it. Let's ask Claude what it can see in this directory.

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: what do you see in this directory?

Claude: I'll help you see what's in the current directory. Let me list the files and directories for you.

```
tool: list_files({})
```

Claude: I can see several files and directories in the current directory: 1. Git-related files and directories: - '.git/' directory with various Git configuration and tracking files - '.gitignore' file 2. Go project files: - 'main.go' - the main Go source code file - 'go.mod' - Go module definition file - 'go.sum' - Go module checksum file 3. Documentation: - 'blogpost.md' - a markdown file, likely containing blog content 4. Environment configuration: - '.envrc' - likely an environment configuration file Would you like me to examine the contents of any specific file from this list?

You:

Works! It can read the directory.

But here's the thing: Claude knows how to combine these tools. We just need to prompt it in a way that provokes it:

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: Tell me about all the Go files in here. Be brief!

Claude: I'll examine the Go files in the current directory for you. Let me check what files are available first.

```
tool: list_files({})
```

Claude: I found a Go file in the current directory. Let me examine it:

```
tool: read_file({"path": "main.go"})
```

```
tool: read_file({"path": "go.mod"})
```

Claude: Here's a brief overview of the Go files in this project: ****main.go****: -
 ↪ Implements a Claude AI agent that can interact with the local filesystem - Uses
 ↪ the Anthropic SDK for Go to communicate with Claude AI models - Provides two
 ↪ tools: `\`read_file\`` and `\`list_files\`` that allow Claude to read file contents
 ↪ and list directory contents - Manages a conversation flow between the user and
 ↪ Claude, handling tool execution when Claude needs to access files ****go.mod****: -
 ↪ Defines the module dependencies - Main dependencies are the Anthropic SDK for
 ↪ Go and a JSON schema library - Uses Go 1.24.1 This appears to be a simple
 ↪ terminal-based application that allows users to chat with Claude while giving
 ↪ the AI access to read local files and list directories.

You:

First it used the `list_files` and then it called `read_file` twice with the Go-related files that I asked it about.

Just...just like we would, right? I mean, here, what would you do if I ask you what version of Go we use in this project? Here's what Claude does for me:

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: What go version are we using in this project?

Claude: I'll help you find the Go version being used in this project. Let me check the files to locate where the Go version might be specified.

```
tool: list_files({})
```

Claude: I can see there's a Go project here with several important files like 'go.mod', 'go.sum', and 'main.go'. Let me check the 'go.mod' file first, as it typically contains the Go version information.

```
tool: read_file({"path": "go.mod"})
```

Claude: According to the 'go.mod' file, this project is using Go version 1.24.1.

Claude looks at the directory, looks at `go.mod`, and has the answer.

We're at around 190 lines of code now. Let that sink in. Once you have, let's add another tool.

Let it edit_file

The last tool we're going to add is `edit_file` — a tool that lets Claude edit files.

"Holy shit", you're thinking now, "this is where the rubber hits the road, this is where he pulls the rabbit out of the hat." Well, let's see, shall we?

First, let's add a definition for our new `edit_file` tool:

```
// main.go

var EditFileDefinition = ToolDefinition{
    Name: "edit_file",
    Description: `Make edits to a text file.

Replaces 'old_str' with 'new_str' in the given file. 'old_str' and 'new_str' MUST
↪ be different from each other.

If the file specified with path doesn't exist, it will be created.
`,
    InputSchema: EditFileInputSchema,
```

```

    Function:    EditFile,
}

type EditFileInput struct {
    Path    string `json:"path" jsonschema_description:"The path to the file"`
    OldStr   string `json:"old_str" jsonschema_description:"Text to search for – must
↪ match exactly and must only have one match exactly"`
    NewStr   string `json:"new_str" jsonschema_description:"Text to replace old_str
↪ with"`
}

var EditFileInputSchema = GenerateSchema[EditFileInput]()

```

That's right, I again know what you're thinking: "string replacement to edit files?" Claude 3.7 loves replacing strings (experimentation is how you find out what they love or don't), so we're going to implement `edit_file` by telling Claude it can edit files by replacing existing text with new text.

Now here's the implementation of the `EditFile` function in Go:

```

func EditFile(input json.RawMessage) (string, error) {
    editFileInput := EditFileInput{}
    err := json.Unmarshal(input, &editFileInput)
    if err != nil {
        return "", err
    }

    if editFileInput.Path == "" || editFileInput.OldStr == editFileInput.NewStr {
        return "", fmt.Errorf("invalid input parameters")
    }

    content, err := os.ReadFile(editFileInput.Path)
    if err != nil {
        if os.IsNotExist(err) && editFileInput.OldStr == "" {
            return createNewFile(editFileInput.Path, editFileInput.NewStr)
        }
        return "", err
    }

    oldContent := string(content)
    newContent := strings.Replace(oldContent, editFileInput.OldStr,
↪ editFileInput.NewStr, -1)

    if oldContent == newContent && editFileInput.OldStr != "" {
        return "", fmt.Errorf("old_str not found in file")
    }

    err = os.WriteFile(editFileInput.Path, []byte(newContent), 0644)

```

```

    if err != nil {
        return "", err
    }

    return "OK", nil
}

```

It checks the input parameters, it reads the file (or creates it if it exists), and replaces the `OldStr` with `NewStr`. Then it writes the content back to disk and returns `"OK"`.

What's missing still is `createNewFile`, which is just a tiny helper function that would be 70% shorter if this wasn't Go:

```

func createNewFile(filePath, content string) (string, error) {
    dir := path.Dir(filePath)
    if dir != "." {
        err := os.MkdirAll(dir, 0755)
        if err != nil {
            return "", fmt.Errorf("failed to create directory: %w", err)
        }
    }

    err := os.WriteFile(filePath, []byte(content), 0644)
    if err != nil {
        return "", fmt.Errorf("failed to create file: %w", err)
    }

    return fmt.Sprintf("Successfully created file %s", filePath), nil
}

```

Last step: adding it to the list of tools that we send to Claude.

```

// main.go

func main() {
    // [... previous code ...]

    tools := []ToolDefinition{ReadFileDefinition, ListFilesDefinition,
↵ EditFileDefinition}

    // [... previous code ...]
}

```

And...we're ready, but are *you*? Are you ready to let it rip?

Thought so, let's do this. Let's tell Claude to create a new FizzBuzz function in JavaScript.

\$ go run main.go

Chat with Claude (use 'ctrl-c' to quit)

You: hey claude, create fizzbuzz.js that I can run with Nodejs and that has fizzbuzz in it and executes it

Claude: I'll create a fizzbuzz.js file that you can run with Node.js. First, let me check if the file already exists:

tool: list_files({})

Claude: I don't see a fizzbuzz.js file, so I'll create one with a classic FizzBuzz implementation. This program will print numbers from 1 to 100, but for multiples of 3 it will print "Fizz", for multiples of 5 it will print "Buzz", and for numbers that are multiples of both 3 and 5 it will print "FizzBuzz".

```
tool: edit_file({"path":"fizzbuzz.js","old_str":"","new_str":"/**\n * FizzBuzz
  ↪ implementation in JavaScript\n * Prints numbers from 1 to 100, but:\n * - For
  ↪ multiples of 3, print \\Fizz\\ instead of the number\n * - For multiples of 5,
  ↪ print \\Buzz\\ instead of the number\n * - For multiples of both 3 and 5, print
  ↪ \\FizzBuzz\\ \n */\n\nfunction fizzBuzz(n) {\n  for (let i = 1; i <= n; i++) {\n
  ↪ let output = '';\n  \n  if (i % 3 === 0) output += 'Fizz';\n  if (i % 5 === 0)
  ↪ output += 'Buzz';\n  \n  console.log(output || i);\n  }\n}\n\n// Execute
  ↪ FizzBuzz for numbers 1 to 100\nconsole.log('Running FizzBuzz:');
  ↪ \nfizzBuzz(100);"}
  ↪
```

Claude: I've created a fizzbuzz.js file that you can run with Node.js. The program: 1. Defines a function called 'fizzBuzz' that takes a parameter 'n' for the upper limit 2. Loops from 1 to n (set to 100 in the execution) 3. For each number: - If it's divisible by 3, it adds "Fizz" to the output - If it's divisible by 5, it adds "Buzz" to the output - If it's divisible by both, it adds both resulting in "FizzBuzz" - If it's not divisible by either 3 or 5, it outputs the number itself 4. The script automatically executes the function for numbers 1 to 100 You can run this script using Node.js with the following command in your terminal: node fizzbuzz.js This will output the FizzBuzz sequence from 1 to 100 to your console.

Right?! It's impressive, isn't it? And that's the *most basic implemenation* of `edit_file` —of an agent in general—you can probably come up with.

But, did it work? Yes, it did:

```
$ node fizzbuzz.js
Running FizzBuzz:
1
2
Fizz
4
Buzz
Fizz
7
8
```

```

Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
[...]

```

Amazing. But hey, let's have it actually *edit* a file and not just create one.

Here's what Claude does when I ask it to "Please edit the `fizzbuzz.js` so that it only prints until 15":

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: Please edit `fizzbuzz.js` so that it only prints until 15

Claude: I'll edit the `fizzbuzz.js` file to print only until 15 instead of its current limit. First, let me check the current content of the file to understand what changes are needed.

```
tool: read_file({"path":"fizzbuzz.js"})
```

Claude: I can see that the file currently runs FizzBuzz up to 100. I'll modify it to only run up to 15 instead by changing the parameter in the `fizzBuzz` function call.

```

tool: edit_file({"path":"fizzbuzz.js","old_str":"// Execute FizzBuzz for numbers 1
↪ to 100\nconsole.log('Running FizzBuzz:');\nfizzBuzz(100);","new_str":"//
↪ Execute FizzBuzz for numbers 1 to 15\nconsole.log('Running FizzBuzz:');
↪ \nfizzBuzz(15);"})

```

Claude: I also need to update the comment at the top of the file to reflect this change:

```

tool: edit_file({"path":"fizzbuzz.js","old_str":"/*\n * FizzBuzz implementation in
↪ JavaScript\n * Prints numbers from 1 to 100, but:","new_str":"/*\n * FizzBuzz
↪ implementation in JavaScript\n * Prints numbers from 1 to 15, but:"})

```

Claude: The changes have been successfully made to the `fizzbuzz.js` file. The program will now: 1. Only print FizzBuzz from 1 to 15 (instead of 1 to 100) 2. The documentation has been updated to reflect this change The function itself remains the same, as it takes a parameter to determine the upper limit, and we've simply changed that parameter from 100 to 15.

It reads the file, it edits the file to change the how long it runs, *and* then it also edits the file to update the comment at the top.

And it still works:

```
$ node fizzbuzz.js
Running FizzBuzz:
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Okay, let's do one more and ask it to do the following:

Create a `congrats.js` script that rot13-decodes the following string 'Pbatenghyngvbaf ba ohvyqvat n pbqr-rqvgvat ntrag!' and prints it

Maybe a tall order. Let's see:

```
$ go run main.go
```

Chat with Claude (use 'ctrl-c' to quit)

You: Create a `congrats.js` script that rot13-decodes the following string 'Pbatenghyngvbaf ba ohvyqvat n pbqr-rqvgvat ntrag!' and prints it

Claude: I'll create a `congrats.js` script that decodes the ROT13-encoded string and prints it. ROT13 is a simple letter substitution cipher that replaces a letter with the 13th letter after it in the alphabet. Let me create this file for you:

```
tool: edit_file({"path":"congrats.js","old_str":"","new_str":"/*\n * A script to
↪ decode a ROT13-encoded string and print the result\n */\n\nfunction
↪ rot13Decode(encodedStr) {\n return encodedStr.replace(/[a-zA-Z]/g,
↪ function(char) {\n // Get the character code\n const charCode =
↪ char.charCodeAt(0);\n \n // Handle uppercase letters\n if (charCode \>= 65 &&
↪ charCode \<= 90) {\n return String.fromCharCode(((charCode - 65 + 13) % 26) +
↪ 65);\n }\n \n // Handle lowercase letters\n if (charCode \>= 97 && charCode \<=
↪ 122) {\n return String.fromCharCode(((charCode - 97 + 13) % 26) + 97);\n }\n \n
↪ // Return non-alphabetic characters as is\n return char;\n });\n}\n\nconst
↪ encodedMessage = 'Pbatenghyngvbaf ba ohvyqvat n pbqr-rqvgvat ntrag!';\nconst
↪ decodedMessage = rot13Decode(encodedMessage);
↪ \n\nconsole.log(decodedMessage);"}})
```

Claude: I've created a JavaScript file called `congrats.js` that: 1. Defines a `'rot13Decode'` function that takes an encoded string and applies the ROT13 cipher to decode it 2. Takes the provided string `'Pbatenghyngvbaf ba ohvyqvat n pbqr-rqvgvat ntrag!'` 3. Decodes it using the function 4. Prints the decoded message to the console You can run this script using Node.js by typing `'node congrats.js'` in your terminal. The script will decode and display the hidden congratulatory message!

Does it work? Let's try it:

```
$ node congrats.js
Congratulations on building a code-editing agent!
```

It does!

Isn't this amazing?

If you're anything like all the engineers I've talked to in the past few months, chances are that, while reading this, you have been waiting for the rabbit to be pulled out of the hat, for me to say "well, in reality it's much, much harder than this." But it's not.

This is essentially all there is to the inner loop of a code-editing agent. Sure, integrating it into your editor, tweaking the system prompt, giving it the right feedback at the right time, a nice UI around it, better tooling around the tools, support for multiple agents, and so on—we've built all of that in **Amp**, but it didn't require moments of genius. All that was required was practical engineering and elbow grease.

These models are incredibly powerful now. 300 lines of code and three tools and now you're to be able to talk to an alien intelligence that edits your code. If you think "well, but we didn't really..."—go and try it! Go and see how far you can get with this. I bet it's a lot farther than you think.

That's why we think everything's changing.

附录 B ·Medium Python 移植版页面存档

以下页面为浏览器打印存档，内容与版权归原作者所有，仅供个人离线阅读。

项	内容
标题	How to Build an Agent by Thorsten Ball （ Python Version ）
作者	Janitha Rathnayake
发布	2025-08-19
地址	https://medium.com/@jbrathnayake98/how-to-build-an-agent-by-thorsten-ball-python-version-ebbabb8665f6
存档方式	Chrome 打印为 PDF （关闭页眉页脚）， 2026-09-05
范围	正文全文 21 页；原页面末尾的广告与推荐位未收录

该页是 Thorsten Ball 原文（附录 A）的 Python 改编版，作者在开头已声明改编关系：保留原文结构与叙述，把 Go 代码换成 Python 。

阅读提示：其中工具 schema 部分使用了 `from jsonschema import generate_schema`，该函数在 `jsonschema` 包中并不存在，照抄会 `ImportError`；模型 ID `claude-3-7-sonnet-latest` 亦已过期。详见第八章「实测勘误」。可运行的 Python 实现见第七章。

How to Build an Agent by Thorsten Ball | (Python Version)

20 min read · Aug 19, 2025



Janitha Rathnayake

Follow



Listen



Share



More

This article, “**How to Build an Agent**,” was originally written by **Thorsten Ball** and published on April 15, 2025, at [amrcode.com/how-to-build-an-agent]. The content has been adapted to replace **Go** code examples with **Python**, but the structure and narrative remain true to the original.

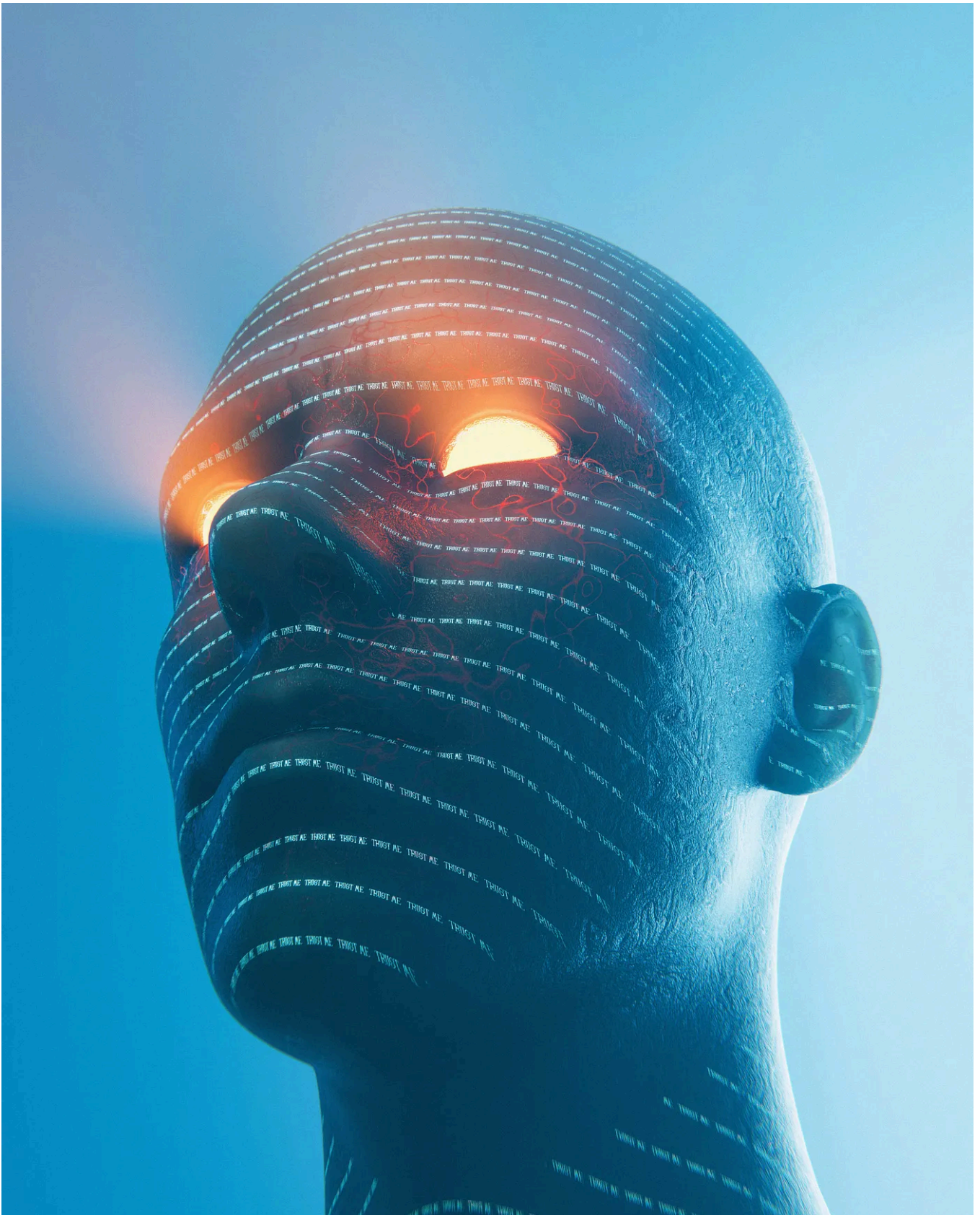


Photo by [Shubham Dhage](#) on [Unsplash](#)

It's not that hard to build a fully functioning, code-editing agent.

It seems like it would be. When you look at an agent editing files, running commands, wriggling itself out of errors, retrying different strategies — it seems like there has to be a

secret behind it.

There isn't. It's an LLM, a loop, and enough tokens. It's what we've been saying on the podcast from the start. The rest, the stuff that makes Amp so addictive and impressive? Elbow grease.

But building a small and yet highly impressive agent doesn't even require that. You can do it in less than 400 lines of code, most of which is boilerplate.

I'm going to show you how, right now. We're going to write some code together and go from zero lines of code to "oh wow, this is... a game changer."

I urge you to follow along. No, really. You might think you can just read this and that you don't have to type out the code, but it's less than 400 lines of code. I need you to feel how little code it is, and I want you to see this with your own eyes in your terminal in your folders.

Here's what we need:

Pencils out!

Let's dive right in and get ourselves a new Python project set up in four easy commands:

```
mkdir code-editing-agent
cd code-editing-agent
python -m venv venv
source venv/bin/activate # On Windows, use: venv\Scripts\activate
touch main.py
```

Now, let's open `main.py` and, as a first step, put a skeleton of things we need in it:

```
import os
import anthropic

class Agent:
    def __init__(self, client, get_user_message):
        self.client = client
        self.get_user_message = get_user_message
```

```

def main():
    client = anthropic.Anthropic()

    def get_user_message():
        user_input = input()
        return user_input, bool(user_input)

    agent = Agent(client, get_user_message)
    agent.run()

if __name__ == "__main__":
    main()

```

Yes, this doesn't do much yet. But what we have here is an `Agent` that has access to an `anthropic.Anthropic` client (which, by default, looks for `ANTHROPIC\_API\_KEY`) and that can get a user message by reading from stdin on the terminal.

Now let's add the missing `run` method:

```

import os
import anthropic

class Agent:
    def __init__(self, client, get_user_message):
        self.client = client
        self.get_user_message = get_user_message

    def run(self):
        conversation = []
        print("Chat with Claude (use 'ctrl-d' to quit)")

        while True:
            print("\033[94mYou\033[0m: ", end="")
            user_input, ok = self.get_user_message()
            if not ok:
                break

            user_message = {"role": "user", "content": [{"type": "text", "text": user_input}]}
            conversation.append(user_message)

            message = self.run_inference(conversation)
            conversation.append({"role": "assistant", "content": message.content})

            for content in message.content:
                if content.type == "text":
                    print(f"\033[93mClaude\033[0m: {content.text}")

    def run_inference(self, conversation):
        message = self.client.messages.create(
            model="claude-3-7-sonnet-latest",

```

```

        max_tokens=1024,
        messages=conversation
    )
    return message

def main():
    client = anthropic.Anthropic()

    def get_user_message():
        try:
            user_input = input()
            return user_input, bool(user_input)
        except EOFError:
            return "", False

    agent = Agent(client, get_user_message)
    agent.run()

if __name__ == "__main__":
    main()

```

It's not a lot, is it? About 40 lines, and the most important thing in them is this loop in `run` that lets us talk to Claude. But that's already the heartbeat of this program.

And for a heartbeat, it's pretty straightforward: we first print a prompt, ask the user to type something, add it to the conversation, send it to Claude, add Claude's response to the conversation, print the response, and around we go.

This is every AI chat application you've ever used, except it's in the terminal.

Let's run it:

```

export ANTHROPIC_API_KEY="this is the last time i will tell you to set this"

# Install the dependencies
pip install anthropic

# Run it
python main.py

```

Then you can just talk to Claude, like this:

```
$ python main.py
```

```
Chat with Claude (use 'ctrl-d' to quit)
```

```
You: Hey! I'm Thorsten! How are you?
```

```
Claude: Hi Thorsten! I'm doing well, thanks for asking. It's nice to meet you. How are you?
```

```
You: Can you come up with any horse-related nicknames that make fun of my first name?
```

```
Claude: I can try to come up with some playful horse-related nicknames based on "Thorsten".
```

```
You:
```

Notice how we kept the same conversation going over multiple turns. It remembers my name from the first message. The `conversation` grows longer with every turn, and we send the whole conversation every time. The server — Anthropic's server — is stateless. It only sees what's in the `conversation` list. It's up to us to maintain that.

Okay, let's move on, because the nicknames suck, and this is not an agent yet. What's an agent? Here's my definition: an LLM with access to tools, giving it the ability to modify something outside the context window.

A First Tool

An LLM with access to tools? What's a tool? The basic idea is this: you send a prompt to the model that says it should reply in a certain way if it wants to use "a tool". Then you, as the receiver of that message, "use the tool" by executing it and replying with the result. That's it. Everything else we'll see is just abstraction on top of it.

Imagine you're talking to a friend and you tell them: "in the following conversation, wink if you want me to raise my arm". Weird thing to say, but an easy concept to grasp.

We can already try it without changing any of our code.

```
$ python main.py
```

```
Chat with Claude (use 'ctrl-d' to quit)
```

```
You: You are a weather expert. When I ask you about the weather in a given location,
```

```
Claude: I understand your instructions. When you ask me about the weather in a specific
```

You: Hey, what's the weather in Munich?

Claude: `get_weather(Munich)`

We told Claude to wink with ``get_weather`` when it wants to know about the weather. T

You: hot and humid, 28 degrees celcius

Claude: Thank you for providing that information. The current weather in Munich is h

That worked very well, on first try, didn't it?

These models are trained and fine-tuned to use “tools” and they're very eager to do so. By now, 2025, they kinda “know” that they don't know everything and can use tools to get more information. (Of course that's not precisely what's going on, but it's good enough an explanation for now.)

To summarize, all there is to tools and tool use are two things:

0. You tell the model what tools are available

1. When the model wants to execute the tool, it tells you, you execute the tool and send the response up

To make (1) easier, the big model providers have built-in APIs to send tool definitions along.

Okay, now let's build our first tool: ``read_file``

The ``read_file`` tool

In order to define the ``read_file`` tool, we're going to use the types that the Anthropic SDK suggests, but keep in mind: under the hood, this will all end up as strings that are sent to the model. It's all “wink if you want me to use ``read_file``”.

Each tool we're going to add will require the following:

- A name
- A description to tell the model what the tool does, when to use it, when to not use it, what it returns and so on

- An input schema that describes, as a JSON schema, what inputs this tool expects and in which form
- A function that actually executes the tool with the input the model sends to us and returns the result

So let's add that to our code:

```
import os
import json
import anthropic
from jsonschema import generate_schema

class ToolDefinition:
    def __init__(self, name, description, input_schema, function):
        self.name = name
        self.description = description
        self.input_schema = input_schema
        self.function = function
```

Now we give our `Agent` tool definitions:

```
class Agent:
    def __init__(self, client, get_user_message, tools=None):
        self.client = client
        self.get_user_message = get_user_message
        self.tools = tools or []

def main():
    client = anthropic.Anthropic()

    def get_user_message():
        try:
            user_input = input()
            return user_input, bool(user_input)
        except EOFError:
            return "", False

    tools = []
    agent = Agent(client, get_user_message, tools)
    agent.run()
```

And send them along to the model in `run\_inference`:

```
def run_inference(self, conversation):
    tools = [
        {
            "name": tool.name,
            "description": tool.description,
            "input_schema": tool.input_schema
        } for tool in self.tools
    ]

    message = self.client.messages.create(
        model="claude-3-7-sonnet-latest",
        max_tokens=1024,
        messages=conversation,
        tools=tools
    )
    return message
```

There's some type handling going on, but it's very simple: we send along our tool definitions, on the server Anthropic wraps these definitions in a system prompt (which isn't much), which it adds to our `conversation`, and the model then replies in a specific way if it wants to use that tool.

Alright, so tool definitions are being sent along, but we haven't defined a tool yet. Let's do that and define `read\_file`:

```
def read_file(input_data):
    input_dict = json.loads(input_data)
    path = input_dict["path"]

    try:
        with open(path, 'r') as file:
            content = file.read()
        return content, None
    except Exception as e:
        return "", str(e)

read_file_definition = ToolDefinition(
    name="read_file",
    description="Read the contents of a given relative file path. Use this when you",
    input_schema=generate_schema({"path": str}, descriptions={"path": "The relative"},
    function=read_file
)
```

That's not much, is it? It's a single function, `read\_file`, and two descriptions the model will see: our `description` that describes the tool itself ("Read the contents of a given

relative file path. ...”) and a description of the single input parameter this tool has (“The relative path of a ...”).

The `input\_schema` stuff? We need that so that we can generate a JSON schema for our tool definition which we send to the model. To do that, we use the `jsonschema` package, which we need to install:

```
pip install jsonschema
```

Then, in the `main` function, we need to make sure that we use the definition:

```
def main():
    client = anthropic.Anthropic()

    def get_user_message():
        try:
            user_input = input()
            return user_input, bool(user_input)
        except EOFError:
            return "", False

    tools = [read_file_definition]
    agent = Agent(client, get_user_message, tools)
    agent.run()
```

Time to try it!

```
$ python main.py
```

Chat with Claude (use 'ctrl-d' to quit)

You: what's in main.py?

Claude: I'll help you check what's in the main.py file. Let me read it for you.

You:

Wait, what? Ho, ho, ho, it wants to use the tool! Obviously the output will be slightly different for you, but it certainly sounds like Claude knows that it can read files, right?

The problem is that we don't listen! When Claude winks, we ignore it. We need to fix that.



Here, let me show you how to do that in a single, quick, surprisingly-agile-for-my-age move by replacing our `Agent`'s `run` method with this:

```
def run(self):
    conversation = []
    print("Chat with Claude (use 'ctrl-d' to quit)")

    read_user_input = True
    while True:
        if read_user_input:
            print("\033[94mYou\033[0m: ", end="")
            user_input, ok = self.get_user_message()
            if not ok:
                break

            user_message = {"role": "user", "content": [{"type": "text", "text": user_input}]}
            conversation.append(user_message)

            message = self.run_inference(conversation)
            conversation.append({"role": "assistant", "content": message.content})

            tool_results = []
            for content in message.content:
                if content.type == "text":
                    print(f"\033[93mClaude\033[0m: {content.text}")
                elif content.type == "tool_use":
                    result = self.execute_tool(content.id, content.name, json.dumps(content.input_data))
                    tool_results.append(result)

            if not tool_results:
                read_user_input = True
                continue

            read_user_input = False
            conversation.append({"role": "user", "content": tool_results})

    def execute_tool(self, tool_id, name, input_data):
        for tool in self.tools:
            if tool.name == name:
                print(f"\033[92mtool\033[0m: {name}({input_data})")
                result, error = tool.function(input_data)
```

```
        if error:
            return {"type": "tool_result", "tool_use_id": tool_id, "content": error}
        return {"type": "tool_result", "tool_use_id": tool_id, "content": result}
    return {"type": "tool_result", "tool_use_id": tool_id, "content": "tool not found"}
```

Squint and you'll see that it's 90% boilerplate and 10% that matter: when we get a `message` back from Claude, we check whether Claude asked us to execute a tool by looking for `content.type == "tool\_use"`, if so we hand over to `execute\_tool`, lookup the tool by name in our local registry, execute it, return the result. If it's an error, we flip a boolean. That's it.

(Yes, there is a loop in a loop, but it doesn't matter.)

We execute the tool, send the result back up to Claude, and ask again for Claude's response. Truly: that's it. Let me show you.

Mise-en-place, run this:

```
echo 'what animal is the most disagreeable because it always says neigh?' > secret-file.txt
```

That creates a `secret-file.txt` in our directory, containing a mysterious riddle.

In that very same directory, let's run our new tool-using agent, and ask it to look at the file:

```
$ python main.py
```

```
Chat with Claude (use 'ctrl-d' to quit)
```

```
You: Claude, buddy, help me solve the riddle in the secret-file.txt file
```

```
Claude: I'll help you solve the riddle in the secret-file.txt file. Let me first read the file.
```

```
tool: read_file({"path":"secret-file.txt"})
```

```
Claude: Great! I've found the riddle. The answer to the riddle is: **A horse** The riddle was: what animal is the most disagreeable because it always says neigh?
```

```
You:
```

Let's take a deep breath and say it together. Ready? Here we go: holy shit. You just give it a tool and it... uses it when it thinks it'll help solve the task. Remember: we didn't say anything about "if a user asks you about a file, read the file". We also didn't say "if something looks like a filename, figure out how to read it". No, none of that. We say "help me solve the thing in this file" and Claude realizes that it can read the file to answer that and off it goes.

Of course, we can be specific and really nudge it towards a tool, but it basically does it all on its own:

```
$ python main.py
```

```
Chat with Claude (use 'ctrl-d' to quit)
```

```
You: What's going on in main.py? Be brief!
```

```
Claude: I'll check the main.py file to see what's going on.
```

```
tool: read_file({"path":"main.py"})
```

```
Claude: Based on my review, main.py implements a Claude AI assistant agent that can
```

```
You:
```

Spot on. Okay, now that we know how to make Claude use tools, let's add a few more.

The `list\_files` tool

If you're anything like me, the first thing you do when you log into a new computer is to get your bearings by running `ls` — list files.

Let's give Claude the same ability, a tool to list files. And here's the complete implementation of a `list\_files` tool:

```
import os
import json

def list_files(input_data):
    input_dict = json.loads(input_data) if input_data else {}
```

```

path = input_dict.get("path", ".")

try:
    files = []
    for root, dirs, filenames in os.walk(path):
        rel_path = os.path.relpath(root, path) if path != "." else "."
        if rel_path != ".":
            files.append(rel_path + "/")
        for fname in filenames:
            rel_file_path = os.path.join(rel_path, fname) if rel_path != "." else fname
            files.append(rel_file_path)
    return json.dumps(files), None
except Exception as e:
    return "", str(e)

list_files_definition = ToolDefinition(
    name="list_files",
    description="List files and directories at a given path. If no path is provided, list files in the current directory.",
    input_schema=generate_schema({"path": str}, descriptions={"path": "Optional relative path to list files and directories from."}),
    function=list_files
)

```

Nothing fancy here: `list\_files` returns the list of files and directories in the current folder. There's a thousand optimizations we could (and probably should) make if this were a serious effort, but since I just want to show you what's in the wizard's hat, this is fine.

One thing to note: we return a list of strings and we denote directories with a trailing slash. That's not required, it's just something I decided to do. There's no fixed format. Anything goes as long as Claude can make sense of it and whether it can you need to figure out by experimentation. You could also prepend each directory with `directory: ` or return a Markdown document with two headers: `directories` and `files`. There's a ton of options and which one you chose depends on what Claude can make the most sense of, how many tokens it requires, how fast it is to generate and read, and so on.

Here, we just want to create a small `list\_files` tool and the easiest option wins.

Of course we need to tell Claude about `list\_files` too:

```

def main():
    client = anthropic.Anthropic()

    def get_user_message():
        try:
            user_input = input()
            return user_input, bool(user_input)
        except EOFError:
            return "", False

```

```
except EOFError:
    return "", False

tools = [read_file_definition, list_files_definition]
agent = Agent(client, get_user_message, tools)
agent.run()
```

And that's it. Let's ask Claude what it can see in this directory.

```
$ python main.py
```

Chat with Claude (use 'ctrl-d' to quit)

You: what do you see in this directory?

Claude: I'll help you see what's in the current directory. Let me list the files and

tool: list_files({})

Claude: I can see several files and directories in the current directory: 1. Git-rel

You:

Works! It can read the directory.

But here's the thing: Claude knows how to combine these tools. We just need to prompt it in a way that provokes it:

```
$ python main.py
```

Chat with Claude (use 'ctrl-d' to quit)

You: Tell me about all the Python files in here. Be brief!

Claude: I'll examine the Python files in the current directory for you. Let me check

tool: list_files({})

Claude: I found a Python file in the current directory. Let me examine it:

tool: read_file({"path":"main.py"})

Claude: Here's a brief overview of the Python files in this project: ****main.py****: -

You:

First it used the ``list_files`` and then it called ``read_file`` with the Python-related file that I asked it about.

Just... just like we would, right? I mean, here, what would you do if I ask you what version of Python we use in this project? Here's what Claude does for me:

```
$ python main.py
```

```
Chat with Claude (use 'ctrl-d' to quit)
```

```
You: What python version are we using in this project?
```

```
Claude: I'll help you find the Python version being used in this project. Let me check
```

```
tool: list_files({})
```

```
Claude: I can see there's a Python project here with several important files like `main.py`
```

```
tool: read_file({"path":"main.py"})
```

```
Claude: The `main.py` file doesn't explicitly state the Python version, but since it
```

Claude looks at the directory, looks at ``main.py``, and provides an informed response.

We're at around 100 lines of code now. Let that sink in. Once you have, let's add another tool.

Let it ``edit_file``

The last tool we're going to add is ``edit_file`` — a tool that lets Claude edit files.

“Holy shit”, you're thinking now, “this is where the rubber hits the road, this is where he pulls the rabbit out of the hat.” Well, let's see, shall we?

First, let's add a definition for our new ``edit_file`` tool:

```

def edit_file(input_data):
    input_dict = json.loads(input_data)
    path = input_dict["path"]
    old_str = input_dict["old_str"]
    new_str = input_dict["new_str"]

    if not path or old_str == new_str:
        return "", "invalid input parameters"

    try:
        if not os.path.exists(path) and old_str == "":
            os.makedirs(os.path.dirname(path), exist_ok=True)
            with open(path, 'w') as file:
                file.write(new_str)
            return f"Successfully created file {path}", None

        with open(path, 'r') as file:
            content = file.read()

        if old_str not in content and old_str != "":
            return "", "old_str not found in file"

        new_content = content.replace(old_str, new_str)

        with open(path, 'w') as file:
            file.write(new_content)
        return "OK", None
    except Exception as e:
        return "", str(e)

edit_file_definition = ToolDefinition(
    name="edit_file",
    description="""Make edits to a text file.

Replaces 'old_str' with 'new_str' in the given file. 'old_str' and 'new_str' MUST be

If the file specified with path doesn't exist, it will be created.
""",
    input_schema=generate_schema(
        {"path": str, "old_str": str, "new_str": str},
        descriptions={
            "path": "The path to the file",
            "old_str": "Text to search for - must match exactly and must only have c
            "new_str": "Text to replace old_str with"
        }
    ),
    function=edit_file
)

```

That's right, I again know what you're thinking: "string replacement to edit files?" Claude 3.7 loves replacing strings (experimentation is how you find out what they love or don't),

so we're going to implement `edit\_file` by telling Claude it can edit files by replacing existing text with new text.

Last step: adding it to the list of tools that we send to Claude.

```
def main():
    client = anthropic.Anthropic()

    def get_user_message():
        try:
            user_input = input()
            return user_input, bool(user_input)
        except EOFError:
            return "", False

    tools = [read_file_definition, list_files_definition, edit_file_definition]
    agent = Agent(client, get_user_message, tools)
    agent.run()
```

And... we're ready, but are you? Are you ready to let it rip?

Thought so, let's do this. Let's tell Claude to create a new FizzBuzz function in JavaScript.

```
$ python main.py
```

Chat with Claude (use 'ctrl-d' to quit)

You: hey claude, create fizzbuzz.js that I can run with Nodejs and that has fizzbuzz

Claude: I'll create a fizzbuzz.js file that you can run with Node.js. First, let me

tool: list_files({})

Claude: I don't see a fizzbuzz.js file, so I'll create one with a classic FizzBuzz i

tool: edit_file({"path":"fizzbuzz.js","old_str":"","new_str":"/**\n * FizzBuzz imple

Claude: I've created a fizzbuzz.js file that you can run with Node.js. The program:

Right?! It's impressive, isn't it? And that's the most basic implementation of `edit\_file` — of an agent in general — you can probably come up with.

But, did it work? Yes, it did:

```
$ node fizzbuzz.js
Running FizzBuzz:
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
[...]
```

Amazing. But hey, let's have it actually edit a file and not just create one.

Here's what Claude does when I ask it to "Please edit the `fizzbuzz.js` so that it only prints until 15":

```
$ python main.py
```

Chat with Claude (use 'ctrl-d' to quit)

You: Please edit fizzbuzz.js so that it only prints until 15

Claude: I'll edit the fizzbuzz.js file to print only until 15 instead of its current

```
tool: read_file({"path":"fizzbuzz.js"})
```

Claude: I can see that the file currently runs FizzBuzz up to 100. I'll modify it to

```
tool: edit_file({"path":"fizzbuzz.js","old_str":"// Execute FizzBuzz for numbers 1 t
```

Claude: I also need to update the comment at the top of the file to reflect this cha

```
tool: edit_file({"path":"fizzbuzz.js","old_str":"/**\n * FizzBuzz implementation in
```

Claude: The changes have been successfully made to the fizzbuzz.js file. The program

It reads the file, it edits the file to change how long it runs, and then it also edits the file to update the comment at the top.

And it still works:

```
$ node fizzbuzz.js
Running FizzBuzz:
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Okay, let's do one more and ask it to do the following:

Create a `congrats.js` script that rot13-decodes the following string 'Pbatenghyngvbaf ba ohvyqvat n pbqr-rqvgvat ntrag!' and prints it

Maybe a tall order. Let's see:

```
$ python main.py

Chat with Claude (use 'ctrl-d' to quit)

You: Create a congrats.js script that rot13-decodes the following string 'Pbatenghyr
Claude: I'll create a congrats.js script that decodes the ROT13-encoded string and p
tool: edit_file({"path":"congrats.js","old_str":"","new_str":"/**\n * A script to de
Claude: I've created a JavaScript file called congrats.js that: 1. Defines a `rot13D
```

Does it work? Let's try it:

```
$ node congrats.js  
Congratulations on building a code-editing agent!
```

It does!

Isn't this amazing?

If you're anything like all the engineers I've talked to in the past few months, chances are that, while reading this, you have been waiting for the rabbit to be pulled out of the hat, for me to say "well, in reality it's much, much harder than this." But it's not.

This is essentially all there is to the inner loop of a code-editing agent. Sure, integrating it into your editor, tweaking the system prompt, giving it the right feedback at the right time, a nice UI around it, better tooling around the tools, support for multiple agents, and so on — we've built all of that in Amp, but it didn't require moments of genius. All that was required was practical engineering and elbow grease.

These models are incredibly powerful now. 300 lines of code and three tools and now you're able to talk to an alien intelligence that edits your code. If you think "well, but we didn't really..." — go and try it! Go and see how far you can get with this. I bet it's a lot farther than you think.

That's why we think everything's changing.



Follow

Written by Janitha Rathnayake

1 follower · 3 following

Building AI, IoT & backend systems — from LoRa water networks to J.A.R.V.I.S.-like assistants — turning complex tech into elegant solutions.

No responses yet

